Bölüm 9: Liste ADT ve Genelleştirilmiş Listeler

C ile Veri Yapıları ve Algoritmalar

Liste ADT sözleşmesi | Dizi ve bağlı gerçekleme | Genelleştirilmiş liste

Kopyalama · eşitlik · gezinme | Derinlik · ters çevirme | Ağacı listeyle kurmak

Paylaşım ve referans sayma

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

GİRİŞ

Öğrenme Çıktıları

Sözleşme

Liste ADT'sini işlemleri, ön ve son koşullarıyla tanımlamak; gerçeklemeyi arayüzden ayırmak.

Karşılaştırma

Dizi ve bağlı gerçeklemeyi erişim, ekleme, bellek açısından karşılaştırmak.

Genelleştirilmiş liste

Atom ve alt liste içeren listeleri tanımlamak, bellek görünümünü çizmek.

Özyinelemeli yordamlar

Kopyalama, eşitlik, gezinme, derinlik ve ters çevirmeyi özyinelemeli yazmak ve izlemek.

Ağaç gösterimi

Ağacı tek satırlık doğrusal forma çevirmek, yöntemin sınırını açıklamak.

Paylaşım

Paylaşımlı ve özyinelemeli listelerdeki tuzakları, referans sayma ve çöp toplama fikrini açıklamak.

Liste ADT: ne yapar, nasıl yaptığı gizli

```c
typedef struct liste Liste;          // opak tür

Liste *liste_olustur(void);
void   liste_yok_et(Liste *L);
int    liste_uzunluk(const Liste *L);
bool   liste_getir(const Liste *L, int i, int *x);
bool   liste_ekle(Liste *L, int i, int x);  // 0<=i<=n
bool   liste_sil(Liste *L, int i, int *x);  // 0<=i<n
int    liste_ara(const Liste *L, int x);    // yoksa -1
```

Doğrusal liste

L = (a_0, a_1, …, a_{n-1}), n ≥ 0; a_i, a_{i+1}'den önce gelir.

Ön / son koşul

`liste_ekle(L, i, x)` : önce 0 ≤ i ≤ n; sonra x i. konumda, uzunluk n+1, eski a_i, …, a_{n-1} birer konum sağa kaydı.

Opak tür

`struct liste` yalnızca .c dosyasında tanımlı; kullanan kod dizi mi bağ mı bilmez, değiştirmek kolaydır.

Dizi ile gerçekleme (sıralı yerleşim)

```c
#define KAPASITE 100
typedef struct {
    int a[KAPASITE];
    int n;                  // öğe sayısı
} DiziListe;

bool dizi_sil(DiziListe *L, int i, int *x) {
    int k;
    if (i < 0 || i >= L->n) return false;
    *x = L->a[i];
    for (k = i; k < L->n - 1; k++)
        L->a[k] = L->a[k + 1];      // sola kaydır
    L->n--;
    return true;
}
```

Erişim

`a[i]` adresi hesaplanır: **O(1)**.

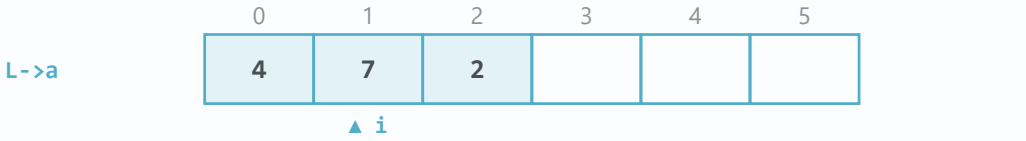
Ekleme / silme

i'den sonraki öğeler kaydırılır: en kötü **O(n)** (başa ekleme, baştan silme).

Kapasite

Sabit dizi dolar; dinamik dizide `realloc` ile iki katına çıkarmak ortalama O(1) ekleme verir.

Kod İzleme: `dizi_ekle(&L, 1, 9)`



Liste [4, 7, 2], `n = 3` ; 9 değeri **1. konuma** eklenecek (0 tabanlı).

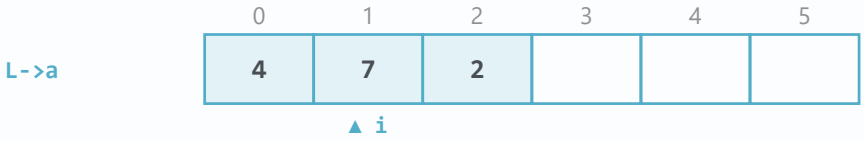
Adım 1 / 9

```
▶ bool dizi_ekle(DiziListe *L, int i, int x) {
2     int k;
3     if (L->n == KAPASITE || i < 0 || i > L->n)
4         return false;          // dolu/geçersiz
5     for (k = L->n; k > i; k--)
6         L->a[k] = L->a[k - 1];     // sağa kaydır
7     L->a[i] = x;
8     L->n++;
9     return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = ?` `L→n = 3` `L→a = [4, 7, 2]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



Denetim: dolu değil ve `0 ≤ i ≤ n` ; ekleme yapılabilir.

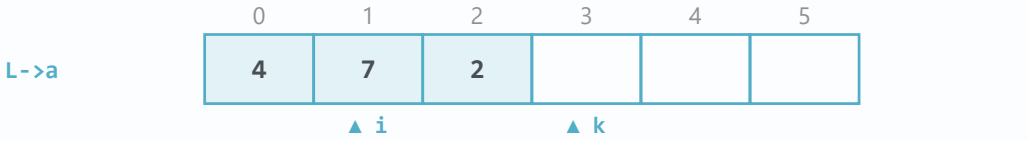
Adım 2 / 9

```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
▶      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
5      for (k = L->n; k > i; k--)
6          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = ?` `L→n = 3` `L→a = [4, 7, 2]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



`k = 3` : `k > i` → doğru, kaydır.

Adım 3 / 9

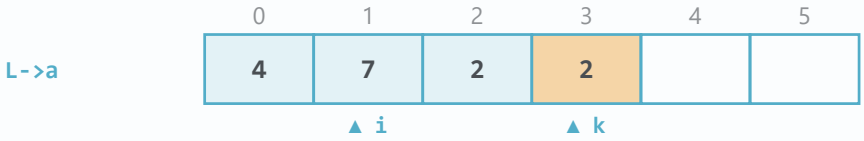
```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
▶      for (k = L->n; k > i; k--)
6          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = 3` `L→n = 3`

`L→a = [4, 7, 2]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



n = 3

`a[3] = a[2]` : 2 bir sağa kaydı.

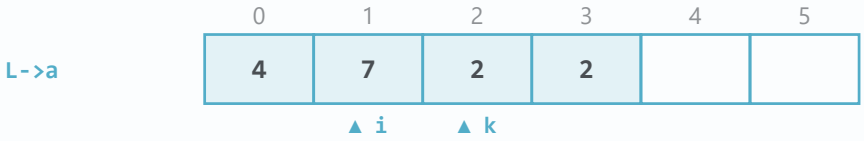
Adım 4 / 9

```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
5      for (k = L->n; k > i; k--)
▶          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = 3` `L→n = 3` `L→a = [4, 7, 2]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



`n = 3`

`k = 2` : `k > i` → doğru, kaydır.

Adım 5 / 9

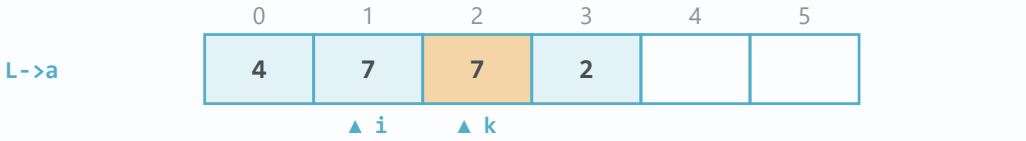
```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
▶      for (k = L->n; k > i; k--)
6          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = 2` `L→n = 3`

`L→a = [4, 7, 2]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



`a[2] = a[1]` : 7 bir sağa kaydı.

Adım 6 / 9

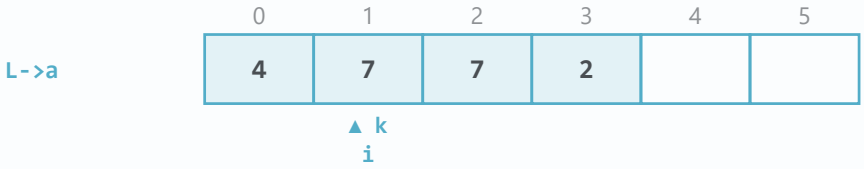
```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
5      for (k = L->n; k > i; k--)
▶          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = 2` `L→n = 3`

`L→a = [4, 7, 7]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



`n = 3`

`k = 1` : `k > i` → yanlış, döngü biter.

Adım 7 / 9

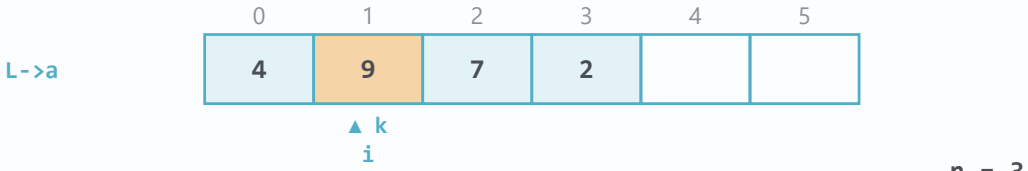
```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
▶      for (k = L->n; k > i; k--)
6          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = 1` `L→n = 3`

`L→a = [4, 7, 7]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



`a[i] = x` : boşalan konuma 9 yazıldı.

Adım 8 / 9

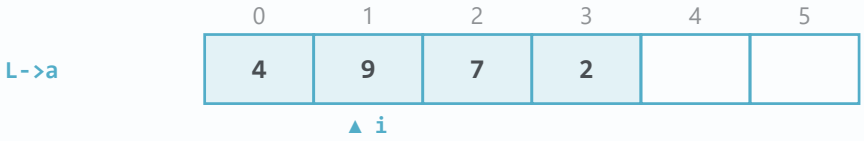
```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
5      for (k = L->n; k > i; k--)
6          L->a[k] = L->a[k - 1];     // sağa kaydır
▶      L->a[i] = x;
8      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = 1` `L→n = 3`

`L→a = [4, 9, 7]`

Kod İzleme: `dizi_ekle(&L, 1, 9)`



`n++` : liste [4, 9, 7, 2]. Kaydırma sayısı n − i: en kötü (i = 0) **O(n)**.

Adım 9 / 9

```
1  bool dizi_ekle(DiziListe *L, int i, int x) {
2      int k;
3      if (L->n == KAPASITE || i < 0 || i > L->n)
4          return false;          // dolu/geçersiz
5      for (k = L->n; k > i; k--)
6          L->a[k] = L->a[k - 1];     // sağa kaydır
7      L->a[i] = x;
▶      L->n++;
9      return true;
10 }
```

DEĞİŞKENLER

`i = 1` `x = 9` `k = ?` `L→n = 4`

`L→a = [4, 9, 7, 2]`

Bağlı gerçekleme: işaretçiye işaretçi

```c
typedef struct dugum *dugum_gost;
struct dugum { int deger; dugum_gost bag; };
typedef struct { dugum_gost bas; int n; } BListe;

bool bl_ekle(BListe *L, int i, int x) {
    dugum_gost *pp = &L->bas, yeni;
    if (i < 0 || i > L->n) return false;
    while (i-- > 0)
        pp = &(*pp)->bag;        // i. bağa yürü
    yeni = malloc(sizeof *yeni);
    if (yeni == NULL) return false;
    yeni->deger = x;
    yeni->bag = *pp;
    *pp = yeni;                  // baş ya da ara: aynı
    L->n++;
    return true;
}
```

`dugum_gost *pp`

Değiştirilecek **bağın adresi**: başta `&L->bas`, sonra `&önceki->bag`. Başa ekleme özel durum olmaktan çıkar.

Maliyet

Konuma yürümek O(i), bağlamak O(1). Kaydırma yok; kapasite sınırı yok.

Uzunluk

`n` alanı tutulursa `liste_uzunluk` O(1).

Dizi mi bağlı liste mi?

Ölçüt	Dizi (sıralı)	Tek yönlü bağlı liste
i. öğeye erişim	O(1)	O(i)
Başa ekleme / silme	O(n) kaydırma	O(1)
Sona ekleme	O(1)* (yer varsa)	O(n), son işaretçisiyle O(1)
Bilinen konumda ekleme	O(n)	O(1) (önceki biliniyorsa)
Arama (sırasız)	O(n)	O(n)
Arama (sıralı)	O(log n) ikili arama	O(n)
Bellek	Yalnız veri; kapasite boşluğu	Düğüm başına işaretçi
Önbellek (cache)	Bitişik: çok iyi	Dağınık: zayıf

Kural: **sık erişim / ikili arama** → dizi; **sık araya ekleme-silme, bilinmeyen boyut** → bağlı liste.

Doğrusal listeden genelleştirilmiş listeye

Doğrusal liste

A = (α_0, …, α_{n-1}): öğelerin her biri **atomik** (dizi, tek yönlü bağlı liste).

Tanım

α_i atomik **ya da bir alt liste** olabiliyorsa A'ya **genelleştirilmiş liste** denir. A listenin adı, n uzunluğudur.

Baş ve kuyruk

α_0 listenin **başı** (head), (α_1, …, α_{n-1}) **kuyruğu** (tail). Kuyruk her zaman bir listedir.

LISP bağlantısı

Baş/kuyruk, LISP'teki `car` / `cdr` 'dir; LISP programları da genelleştirilmiş listedir.

Liste	Uzunluk	Açıklama
D = ()	0	boş liste
A = (a, (b, c))	2	ilk öğe atom, ikinci öğe (b, c) alt listesi; baş(A) = a, kuyruk(A) = ((b, c))
B = (A, A, ())	3	**paylaşımlı** alt liste: ilk iki öğe aynı A; üçüncüsü boş liste
C = (a, C)	2	**özyinelemeli** liste: kendini içerir, C = (a, (a, (a, …)))

Örnek: çok değişkenli polinom

```
typedef enum { VAR, PTR, NO } uclu_t;
typedef struct poge *poge_gost;
struct poge {
    uclu_t uclu;            // üçlü
    union {
        char deg;           // VAR: değişkenin adı
        poge_gost L_bag;    // PTR: katsayı bir liste
        int kat;            // NO : tamsayı katsayı
    };
    int us;                 // üs bilgisi
    poge_gost bag;
};
```

Üç tür öğe

VAR: liste başı, değişken adı (us = 0). **PTR**: katsayı bir alt liste. **NO**: katsayı tam sayı.

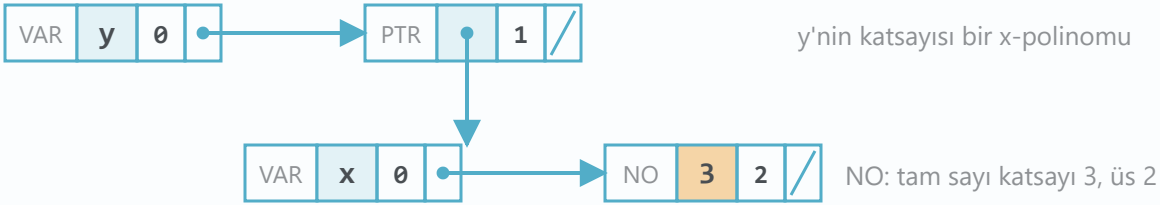
Gruplama

P(x,y,z) önce z'ye göre gruplanır; her z-teriminin katsayısı bir y-polinomu, onunkiler de x-polinomu.

Hocadan "Polinomların genelleştirilmiş liste ile gerçeklemesi için bu, tek bir örnek. Başka şekilde de gerçeklenebilir. Ama bunu yapan dönüşüm yordamlarının da yazılabilmesi gerekir."

Polinomun bellek görünümü

$3x^2y = (3x^2)\cdot y^1$

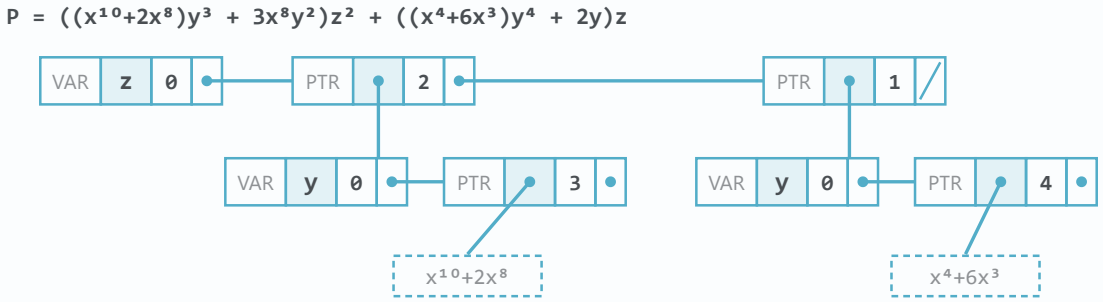


VAR: liste başı, `deg` 'de değişken adı. **PTR**: katsayı `L_bag` 'deki liste. **NO**: katsayı tam sayı.

Adım 1 / 2

Çizimi basitleştirmek için alt düzeyler kısaltıldı.

Polinomun bellek görünümü



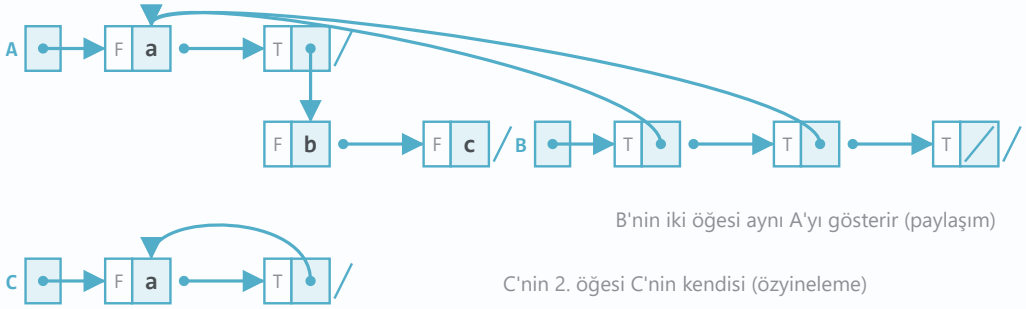
(y-listelerinin kalan öğeleri ve x-listeleri kısaltıldı)

$P(x,y,z) = x^{10}y^3z^2 + 2x^8y^3z^2 + 3x^8y^2z^2 + x^4y^4z + 6x^3y^4z + 2yz$, z'ye göre gruplanıp iç içe listeye yerleşir.

Adım 2 / 2

Çizimi basitleştirmek için alt düzeyler kısaltıldı.

Öğe yapısı ve bellek görünümü




```c
typedef enum { FALSE, TRUE } boolean;
typedef struct oge *oge_gost;
struct oge {
    boolean isaret;       // FALSE: atom
    union {
        char veri;        // atomik değer
        oge_gost L_bag;   // alt listenin başı
    };
    oge_gost bag;         // aynı düzeydeki sonraki
};
```

Öğe

[isaret | veri/L_bag | bag]: F = atom, T = alt liste.

Bu sunumda

Bütün yordamlar bu yapıyı (isaret alanlı öğe) kullanır.

Adsız `union` C11 özelliğidir; C99 için union'a ad verilip `L→u.veri` yazılır.

Neden genelleştirilmiş liste?

Az yer

Ağaç gibi hiyerarşik veri tek satırlık metin kadar yerde saklanır; bir tür sıkıştırma.

Doğal hiyerarşi

Dosya sistemi dizinleri, açılır (popup) menüler, XML/JSON iç içe yapıları, LISP ifadeleri.

Algoritma uzayı

Veriyi iyi bildiğimiz yapıya taşırız; artık tüm liste yordamlarını kullanabiliriz.

Hocadan "Veriyi ağaç olarak tutmak da mümkündür, ama text'le karşılaştırılmaz. "Ben bitmap saklarım. Ağacın bellek görüntüsünü saklarım." Olur, saklanabilir. Ama bu şekilde tutarsak daha az yer kaplıyor... Bu kadar küçük bir verinin saklanmasında bile bilgisayar mühendisi dikkatli olmalıdır."

Hocadan "Bir verinin başka bir uzaya taşınması... Z-transformation... Çok iyi bilinen, hâkim olunan bir uzaya taşınıyor. Biz de veriyi aldık, çok iyi bildiğimiz algoritma uzayına taşıdık."

SORU: paylaşımsız, özyinelemesiz listeyi kopyala

```
oge_gost kopyala(oge_gost L) {
    oge_gost q = NULL;
    if (L) {                    // L NULL değil mi
        q = ogeal();            // bir öğe al
        q->isaret = L->isaret;
        if (!L->isaret) q->veri = L->veri; // atom
        else q->L_bag = kopyala(L->L_bag);
        q->bag = kopyala(L->bag);
    }
    return q;
}
```

Yapı

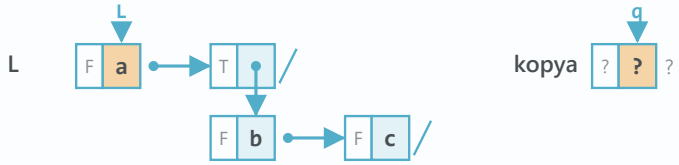
Her öğe için yeni öğe alınır, tür kopyalanır; atomsa değer, değilse alt listenin **kopyası** bağlanır; sonra kalan kopyalanır.

Sınır

Yordam yalnız **paylaşımsız** listelerde doğrudur; paylaşımlı bir alt liste, gösterildiği her yerde ayrı ayrı kopyalanır.

`ogeal()` : `malloc(sizeof *p)` + NULL denetimi yapan yardımcı (bellek yoksa programı durdurur).

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



`ogeal()` : a için yeni öğe ayrıldı (alanlar henüz çöp).

Adım 1 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
▶          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

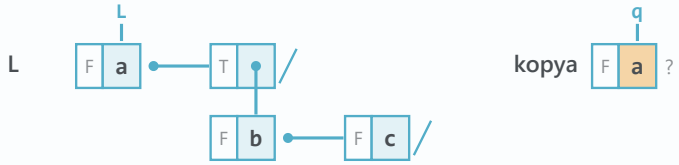
`L = &öğe(a)` `q = &kopya(a)`

ÇAĞRI YIĞINI

`kopyala(a)`

`main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Atomik: `q→isaret = FALSE`, `q→veri = 'a'`.

Adım 2 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
▶          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

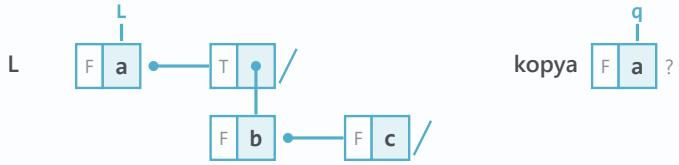
`L = &öğe(a)` `q = &kopya(a)`

ÇAĞRI YIĞINI

`kopyala(a)`

`main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Sıradaki öğe için çağrı: `kopyala(L→bag)`.

Adım 3 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

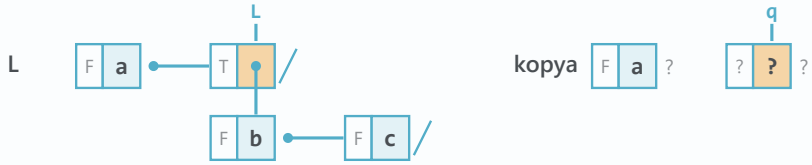
DEĞİŞKENLER

`L = &öğe(a)` `q = &kopya(a)`

ÇAĞRI YIĞINI

- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



`ogeal()` : (b,c) için yeni öğe ayrıldı (alanlar henüz çöp).

Adım 4 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
▶          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

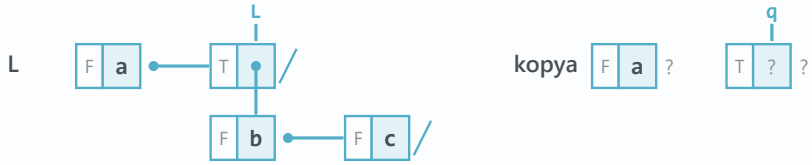
DEĞİŞKENLER

`L = &öğe((b,c))` `q = &kopya((b,c))`

ÇAĞRI YIĞINI

- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Alt liste: önce **alt listenin kopyası** istenir (özyinelemeli çağrı).

Adım 5 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
▶          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

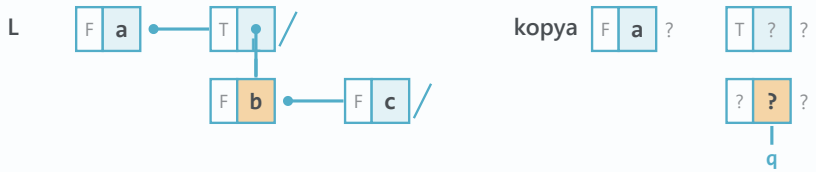
DEĞİŞKENLER

`L = &öğe((b,c))` `q = &kopya((b,c))`

ÇAĞRI YIĞINI

- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



`ogeal()` : b için yeni öğe ayrıldı (alanlar henüz çöp).

Adım 6 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
▶          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

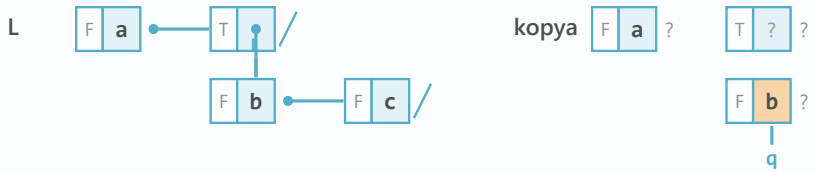
DEĞİŞKENLER

`L = &öğe(b)` `q = &kopya(b)`

ÇAĞRI YIĞINI

- `kopyala(b)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Atomik: `q→isaret = FALSE`, `q→veri = 'b'`.

Adım 7 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
▶          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

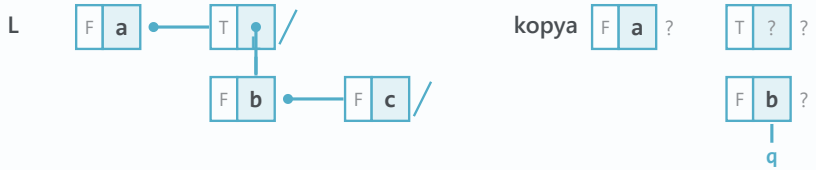
DEĞİŞKENLER

`L = &öğe(b)` `q = &kopya(b)`

ÇAĞRI YIĞINI

- **`kopyala(b)`**
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Sıradaki öğe için çağrı: `kopyala(L→bag)`.

Adım 8 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

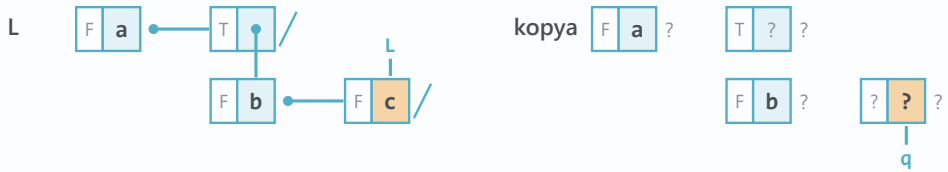
DEĞİŞKENLER

`L = &öğe(b)` `q = &kopya(b)`

ÇAĞRI YIĞINI

- **`kopyala(b)`**
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



`ogeal()` : c için yeni öğe ayrıldı (alanlar henüz çöp).

Adım 9 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
▶          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

`L = &öğe(c)` `q = &kopya(c)`

ÇAĞRI YIĞINI

- `kopyala(c)`
- `kopyala(b)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Atomik: `q→isaret = FALSE`, `q→veri = 'c'`.

Adım 10 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
▶          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

`L = &öğe(c)` `q = &kopya(c)`

ÇAĞRI YIĞINI

- **`kopyala(c)`**
- `kopyala(b)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Sıradaki öğe için çağrı: `kopyala(L→bag)`.

Adım 11 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

`L = &öğe(c)` `q = &kopya(c)`

ÇAĞRI YIĞINI

- **`kopyala(c)`**
- `kopyala(b)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



`kopyala(NULL)` (bag): `L` NULL → hiçbir şey ayırmadan `q = NULL` döner.

Adım 12 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
▶      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

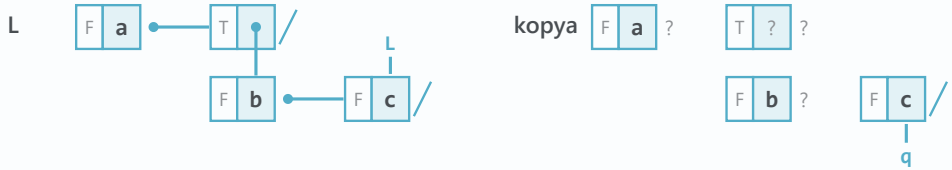
DEĞİŞKENLER

`L = NULL` `q = NULL`

ÇAĞRI YIĞINI

- `kopyala(NULL)`
- `kopyala(c)`
- `kopyala(b)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Dönüş: `q→bag` = NULL (son öğe).

Adım 13 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

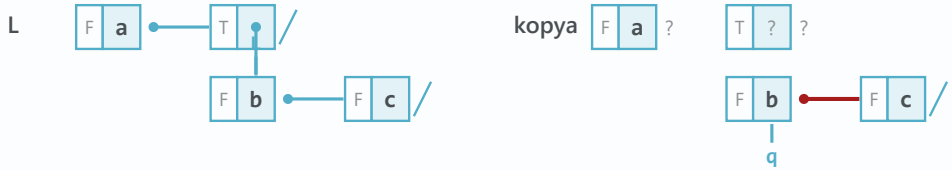
DEĞİŞKENLER

`L = &öğe(c)` `q = &kopya(c)`

ÇAĞRI YIĞINI

- **kopyala(c)**
- kopyala(b)
- kopyala((b,c))
- kopyala(a)
- main()

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Dönüş: `q→bag` ← &kopya(c).

Adım 14 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

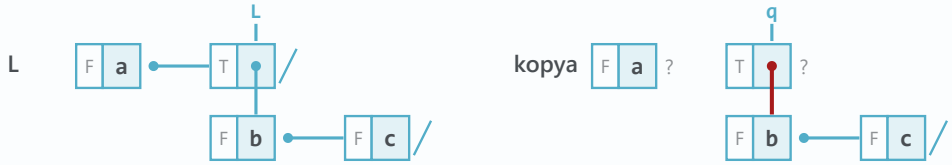
DEĞİŞKENLER

`L = &öğe(b)` `q = &kopya(b)`

ÇAĞRI YIĞINI

- `kopyala(b)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Dönüş: `q→L_bag` ← alt listenin kopyası &kopya(b).

Adım 15 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
▶          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

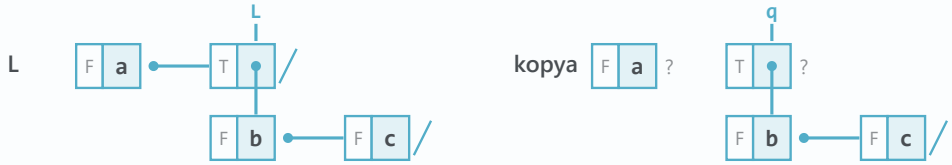
DEĞİŞKENLER

`L = &öğe((b,c))` `q = &kopya((b,c))`

ÇAĞRI YIĞINI

- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Sıradaki öğe için çağrı: `kopyala(L→bag)`.

Adım 16 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

`L = &öğe((b,c))` `q = &kopya((b,c))`

ÇAĞRI YIĞINI

- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



`kopyala(NULL)` (bag): `L` NULL → hiçbir şey ayırmadan `q = NULL` döner.

Adım 17 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
▶      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

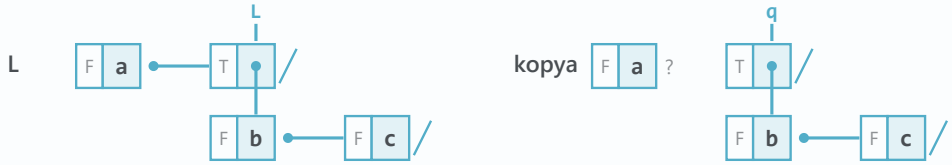
DEĞİŞKENLER

`L = NULL` `q = NULL`

ÇAĞRI YIĞINI

- `kopyala(NULL)`
- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Dönüş: `q→bag` = NULL (son öğe).


```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

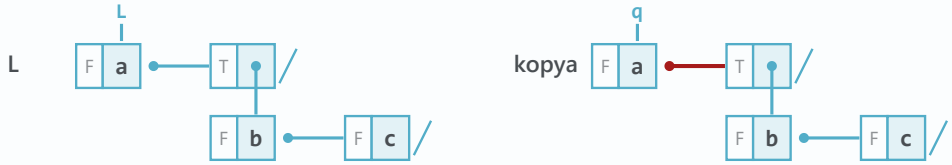
DEĞİŞKENLER

`L = &öğe((b,c))` `q = &kopya((b,c))`

ÇAĞRI YIĞINI

- `kopyala((b,c))`
- `kopyala(a)`
- `main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Dönüş: `q→bag` ← &kopya((b,c)).

Adım 19 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
▶          q->bag = kopyala(L->bag);
9      }
10     return q;
11 }
```

DEĞİŞKENLER

`L = &öğe(a)` `q = &kopya(a)`

ÇAĞRI YIĞINI

`kopyala(a)`

`main()`

Kod İzleme: `kopyala(L)`, L = (a,(b,c))



Kopya tamam: aynı yapı, **yeni** öğeler. Alırken baştan sona, güncellerken sondan başa.

Adım 20 / 20

```
1  oge_gost kopyala(oge_gost L) {
2      oge_gost q = NULL;
3      if (L) {                    // L NULL değil mi
4          q = ogeal();            // bir öğe al
5          q->isaret = L->isaret;
6          if (!L->isaret) q->veri = L->veri; // atom
7          else q->L_bag = kopyala(L->L_bag);
8          q->bag = kopyala(L->bag);
9      }
▶      return q;
11 }
```

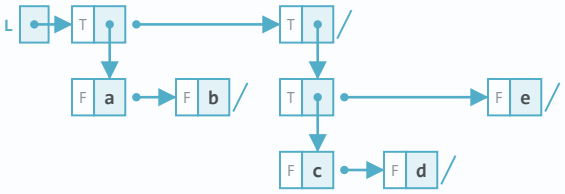
DEĞİŞKENLER

`sonuç = &kopya(a)`

ÇAĞRI YIĞINI

`main()`

Kopyalama: test listesi ve sınırlar



Test için: L = ((a,b),((c,d),e))

Paylaşımlı liste

B = (A,A,()) kopyalanırsa A **iki kez** kopyalanır; kopyada paylaşım kaybolur.

Özyinelemeli liste

C = (a,C) için çağrılar hiç bitmez; yığın taşar ve program çakılır.

Hocadan "Alırken baştan sona, güncellerken sondan başa… "Hocam ben özyinelemeli yazmıycam." diyen var mı hâlâ? "Ben onu kuyruğa koyarım." diyen arkadaş varsa artık bir şey diyemeyeceğim…"

SORU: paylaşımsız, özyinelemesiz iki liste eşit mi?

```
boolean esitmi(oge_gost T, oge_gost L) {
    boolean x = TRUE;
    if (!T && !L) return TRUE;
    if (T && L && T->isaret == L->isaret) {
        if (!T->isaret)         // atomik mi?
            x = (T->veri == L->veri) ? TRUE : FALSE;
        else                    // alt listeler
            x = esitmi(T->L_bag, L->L_bag);
        if (x == TRUE) return esitmi(T->bag, L->bag);
    }
    return FALSE;
}
```

Mantık

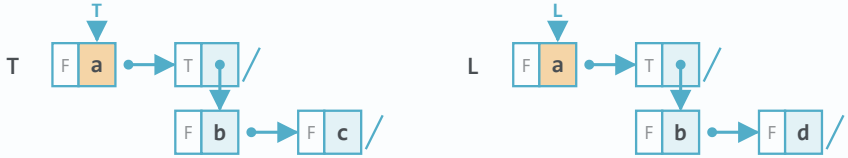
İkisi birlikte biterse eşit. Türler aynıysa: atomda değer, alt listede özyinelemeli eşitlik; ardından kalan öğeler.

Sınır

Özyinelemeli listelerde (C = (a,C)) çağrılar sonsuza gider.

Hocadan "Bu kod, ustaca yazılmış bir kod değil. Daha nitelikli kod yazılabilir. Bir de sonsuz çağrılar sebebiyle özyinelemeli listeler karşılaştırılamaz."

Kod İzleme: `esitmi(T, L)`



İkisi de var ve `isaret` alanları aynı: içerik karşılaştırılır.

Adım 1 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
▶      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

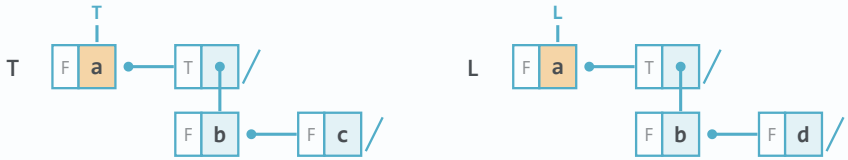
`T = &öğe(a)` `L = &öğe(a)` `x = TRUE`

ÇAĞRI YIĞINI

`esitmi(a, a)`

`main()`

Kod İzleme: `esitmi(T, L)`



Atomik: `'a' == 'a'` → `x = TRUE`.

Adım 2 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
▶              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

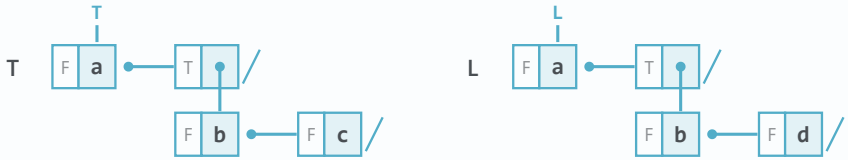
`T = &öğe(a)` `L = &öğe(a)` `x = TRUE`

ÇAĞRI YIĞINI

`esitmi(a, a)`

`main()`

Kod İzleme: `esitmi(T, L)`



x doğru: kalan öğeler karşılaştırılır (`bag` alanları).

Adım 3 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
▶          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

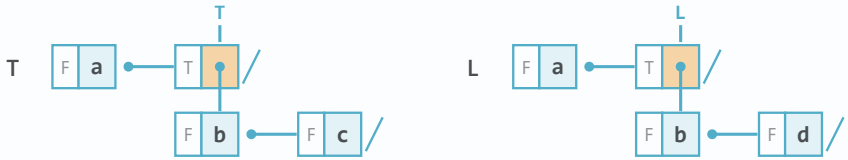
`T = &öğe(a)` `L = &öğe(a)` `x = TRUE`

ÇAĞRI YIĞINI

`esitmi(a, a)`

`main()`

Kod İzleme: `esitmi(T, L)`



İkisi de var ve `isaret` alanları aynı: içerik karşılaştırılır.

Adım 4 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
▶      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

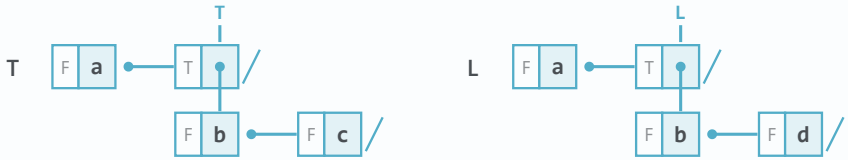
`T = &öğe((b,c))` `L = &öğe((b,d))`

`x = TRUE`

ÇAĞRI YIĞINI

- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



Alt liste: önce alt listeler karşılaştırılır.

Adım 5 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)         // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                    // alt listeler
▶              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe((b,c))` `L = &öğe((b,d))` `x = TRUE`

ÇAĞRI YIĞINI

- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



İkisi de var ve `isaret` alanları aynı: içerik karşılaştırılır.

Adım 6 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
▶      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe(b)` `L = &öğe(b)`

`x = TRUE`

ÇAĞRI YIĞINI

- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



Atomik: `'b' == 'b'` → `x = TRUE` .

Adım 7 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
▶              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe(b)` `L = &öğe(b)` `x = TRUE`

ÇAĞRI YIĞINI

- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



x doğru: kalan öğeler karşılaştırılır (`bag` alanları).

Adım 8 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
▶          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

- `T = &öğe(b)`
- `L = &öğe(b)`
- `x = TRUE`

ÇAĞRI YIĞINI

- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



İkisi de var ve `isaret` alanları aynı: içerik karşılaştırılır.

Adım 9 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
▶      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe(c)` `L = &öğe(d)`

`x = TRUE`

ÇAĞRI YIĞINI

- `esitmi(c, d)`
- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



Atomik: `'c' == 'd'` → `x = FALSE`.

Adım 10 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
▶              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe(c)` `L = &öğe(d)`

`x = FALSE`

ÇAĞRI YIĞINI

- **`esitmi(c, d)`**
- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



x yanlış: `return FALSE` ; kalan öğelere hiç bakılmaz.

Adım 11 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
▶      return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe(c)` `L = &öğe(d)`

`x = FALSE`

ÇAĞRI YIĞINI

- **`esitmi(c, d)`**
- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



Dönüş değeri aynen yukarı iletilir: `FALSE`.

Adım 12 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
▶          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

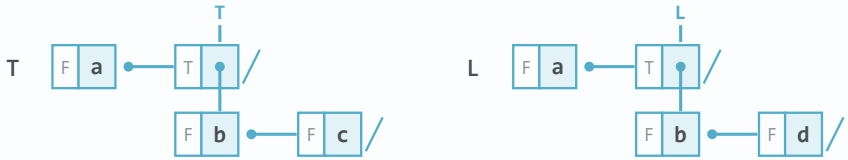
`T = &öğe(b)` `L = &öğe(b)`

`x = TRUE`

ÇAĞRI YIĞINI

- `esitmi(b, b)`
- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



Alt listelerden dönen: `x = FALSE`.

Adım 13 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
▶              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

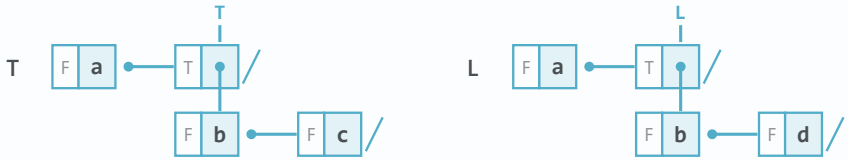
DEĞİŞKENLER

`T = &öğe((b,c))` `L = &öğe((b,d))` `x = FALSE`

ÇAĞRI YIĞINI

- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



x yanlış: `return FALSE` ; kalan öğelere hiç bakılmaz.

Adım 14 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
9          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
▶      return FALSE;
12 }
```

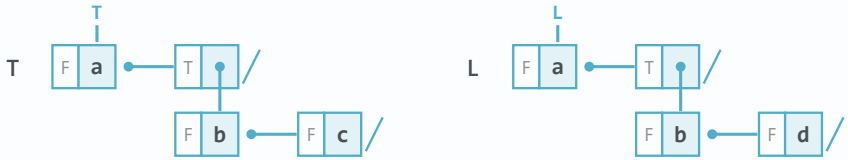
DEĞİŞKENLER

`T = &öğe((b,c))` `L = &öğe((b,d))` `x = FALSE`

ÇAĞRI YIĞINI

- `esitmi((b,c), (b,d))`
- `esitmi(a, a)`
- `main()`

Kod İzleme: `esitmi(T, L)`



Dönüş değeri aynen yukarı iletilir: `FALSE`.

Adım 15 / 15

```
1  boolean esitmi(oge_gost T, oge_gost L) {
2      boolean x = TRUE;
3      if (!T && !L) return TRUE;
4      if (T && L && T->isaret == L->isaret) {
5          if (!T->isaret)          // atomik mi?
6              x = (T->veri == L->veri) ? TRUE : FALSE;
7          else                     // alt listeler
8              x = esitmi(T->L_bag, L->L_bag);
▶          if (x == TRUE) return esitmi(T->bag, L->bag);
10     }
11     return FALSE;
12 }
```

DEĞİŞKENLER

`T = &öğe(a)` `L = &öğe(a)` `x = TRUE`

ÇAĞRI YIĞINI

`esitmi(a, a)`

`main()`

Daha nitelikli sürüm ve paylaşımlı listeler

```
bool esit(oge_gost T, oge_gost L) {
    if (T == L) return true;        // aynı düğüm/ikisi NULL
    if (!T || !L || T->isaret != L->isaret) return false;
    if (T->isaret ? !esit(T->L_bag, L->L_bag)
                  : T->veri != L->veri)
        return false;
    return esit(T->bag, L->bag);
}
```

`T == L` kısayolu

İkisi NULL ise ya da **aynı** (paylaşılan) alt listeyi gösteriyorsa içine inmeden TRUE.

SORU: paylaşımlı listeler

Özyinelemesiz (döngüsüz) paylaşımlı listelerde de aynı yordam doğru sonuç verir; kısayol, ortak alt listeleri tekrar gezmeyi önler.

Hocadan "Bakın, programın direnç noktasını tespit ediyorum. Tuşa basıldı, çalıştı; yetmez! … Hâlbuki ispat yapılırken daha geniş bir bilgi sahasına hâkim olunduğu gösterilir."

SORU: listeyi gez, atomları yaz

Sık görülen hatalı yazım

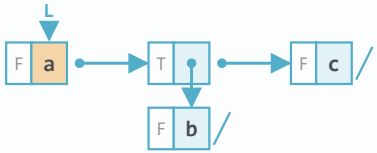
```
void dolas_yaz( oge_gost L ){
    if(L)
        if(!L->isaret)
            printf("%c", L->veri);
        else
            dolas(L->L_bag);
        dolas( L->bag );
}
```

Düzeltilmiş

```
void dolas_yaz(oge_gost L) {
    if (L) {
        if (!L->isaret)
            printf("%c", L->veri); // atomikse yaz
        else
            dolas_yaz(L->L_bag);
        dolas_yaz(L->bag);
    }
}
```

İki hata: (1) Özyinelemeli çağrılar `dolas` adını kullanıyor, yordamın adı `dolas_yaz` (derlenmez). (2) Süslü parantez yok: `if(L)` yalnızca iç `if-else`'i kapsar, 7. satır **`L == NULL` iken de** çalışır → `NULL→bag` ile çöker. Doğrusu iki çağrıyı da `if (L) { … }` içine almaktır.

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Atomik öğe: `'a'` yazıldı.

Adım 1 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
▶             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
7         dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe(a)`

ÇAĞRI YIĞINI

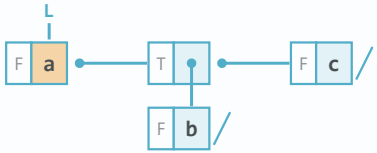
`dolas_yaz(a)`

`main()`

ÇIKTI

```
a
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Aynı düzeyde sıradaki öğeye geçilir (`L→bag`).

Adım 2 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
4             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
▶           dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe(a)`

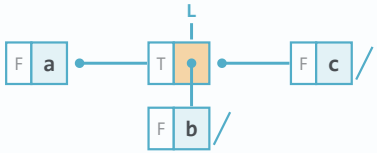
ÇAĞRI YIĞINI

- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
a
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Alt liste: önce içine dalınır (`L→L_bag`).

Adım 3 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
4             printf("%c", L->veri); // atomikse yaz
5         else
▶           dolas_yaz(L->L_bag);
7         dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe((b))`

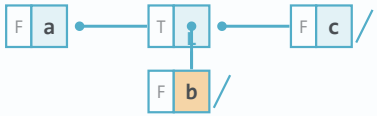
ÇAĞRI YIĞINI

- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
a
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Atomik öğe: `'b'` yazıldı.

Adım 4 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
              printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
7         dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe(b)`

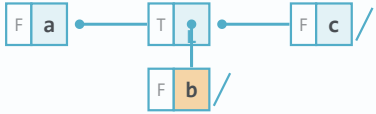
ÇAĞRI YIĞINI

- `dolas_yaz(b)`
- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
ab
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Aynı düzeyde sıradaki öğeye geçilir (`L→bag`).

Adım 5 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
4             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
▶           dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe(b)`

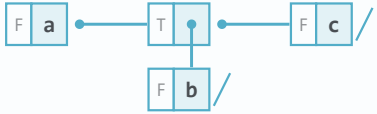
ÇAĞRI YIĞINI

- `dolas_yaz(b)`
- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
ab
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



`L == NULL` (düzeyin sonu): hiçbir şey yapmadan dönülür.

Adım 6 / 11

```
1 void dolas_yaz(oge_gost L) {
▶       if (L) {
3           if (!L->isaret)
4               printf("%c", L->veri); // atomikse yaz
5           else
6               dolas_yaz(L->L_bag);
7           dolas_yaz(L->bag);
8       }
9   }
```

DEĞİŞKENLER

`L = NULL`

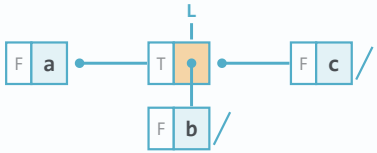
ÇAĞRI YIĞINI

- `dolas_yaz(NULL)`
- `dolas_yaz(b)`
- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
ab
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Aynı düzeyde sıradaki öğeye geçilir (`L→bag`).

Adım 7 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
4             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
▶           dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe((b))`

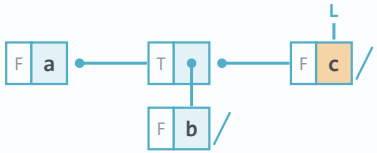
ÇAĞRI YIĞINI

- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
ab
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Atomik öğe: `'c'` yazıldı.

Adım 8 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
▶             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
7         dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe(c)`

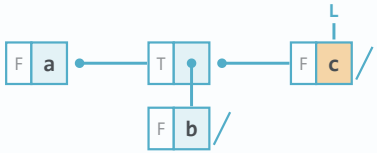
ÇAĞRI YIĞINI

- `dolas_yaz(c)`
- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
abc
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Aynı düzeyde sıradaki öğeye geçilir (`L→bag`).

Adım 9 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
4             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
▶           dolas_yaz(L->bag);
8     }
9 }
```

DEĞİŞKENLER

`L = &öğe(c)`

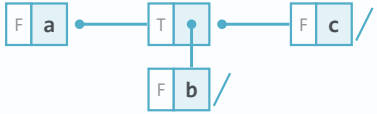
ÇAĞRI YIĞINI

- `dolas_yaz(c)`
- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
abc
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



`L == NULL` (düzeyin sonu): hiçbir şey yapmadan dönülür.

Adım 10 / 11

```
1 void dolas_yaz(oge_gost L) {
▶       if (L) {
3           if (!L->isaret)
4               printf("%c", L->veri); // atomikse yaz
5           else
6               dolas_yaz(L->L_bag);
7           dolas_yaz(L->bag);
8       }
9   }
```

DEĞİŞKENLER

`L = NULL`

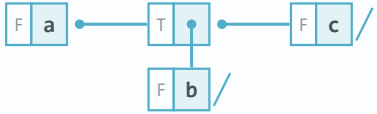
ÇAĞRI YIĞINI

- `dolas_yaz(NULL)`
- `dolas_yaz(c)`
- `dolas_yaz((b))`
- `dolas_yaz(a)`
- `main()`

ÇIKTI

```
abc
```

Kod İzleme: `dolas_yaz(L)`, L = (a,(b),c)



Tüm atomlar soldan sağa, derinlikten bağımsız yazıldı: **abc**. Her öğe bir kez: **O(n)**.

Adım 11 / 11

```
1 void dolas_yaz(oge_gost L) {
2     if (L) {
3         if (!L->isaret)
4             printf("%c", L->veri); // atomikse yaz
5         else
6             dolas_yaz(L->L_bag);
7         dolas_yaz(L->bag);
▶     }
9 }
```

DEĞİŞKENLER

`L = –`

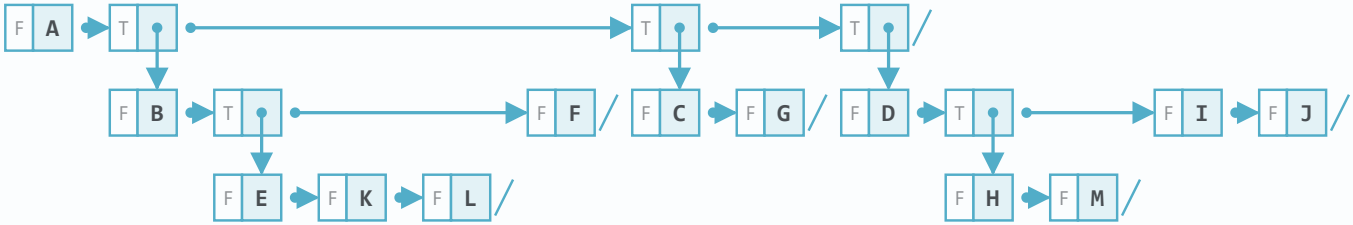
ÇAĞRI YIĞINI

`main()`

ÇIKTI

```
abc
```

Örnek: A(B(E(K,L),F),C(G),D(H(M),I,J))



Her ağaç düğümü bir alt liste: başı düğümün etiketi, kalanı çocukları. Aynı düzeydeki öğeler bir satır boyunca izlenir.

`dolas_yaz` bu liste için **ABEKLFCGDHMIJ** yazar: ağacın kök önde (preorder) gezinmesi.

SORU: listenin derinliği — özyinelemeli

```
int derinlik(oge_gost L) {
    oge_gost p = L;  int m = 0;     // m: en büyük
    if (!L) return 0;
    while (p) {
        if (p->isaret) {
            int n = derinlik(p->L_bag);
            if (m < n) m = n;
        }
        p = p->bag;
    }
    return m + 1;
}
```

Tanım

derinlik(()) = 0; aksi hâlde 1 + alt listelerin en büyük derinliği. Atomlar derinlik katmaz.

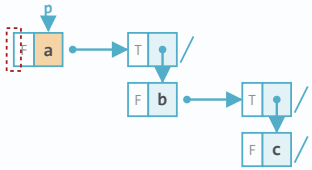
Örnekler

(a, b) → 1 · (a, (b, c)) → 2 · (a, (b, (c))) → 3 · (()) → 1.

Maliyet

Her öğe bir kez ziyaret: O(n); çağrı yığını en çok derinlik kadar.

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



1. düzey çağrı: `p = L` , `m = 0` .

Adım 1 / 16

```
1  int derinlik(oge_gost L) {
▶      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

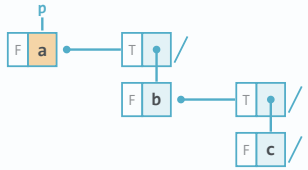
DEĞİŞKENLER

`p = &öğe(a)` `m = 0`

ÇAĞRI YIĞINI

- **`derinlik(a)`**
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



'a' atomik: derinliği artırmaz.

Adım 2 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
▶          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

DEĞİŞKENLER

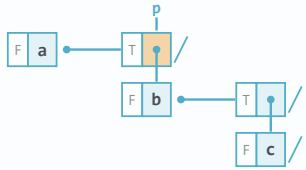
`p = &öğe(a)` `m = 0`

ÇAĞRI YIĞINI

`derinlik(a)`

`main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



Alt liste bulundu: derinliği özyinelemeli olarak hesaplanır.


```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
▶              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

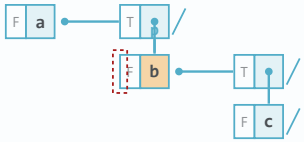
DEĞİŞKENLER

`p = &öğe((b,(c)))` `m = 0`

ÇAĞRI YIĞINI

- `derinlik(a)`
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



2. düzey çağrı: `p = L` , `m = 0` .

Adım 4 / 16

```
1  int derinlik(oge_gost L) {
▶      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

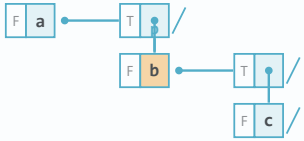
DEĞİŞKENLER

`p = &öğe(b)` `m = 0`

ÇAĞRI YIĞINI

- `derinlik(b)`
- `derinlik(a)`
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



'b' atomik: derinliği artırmaz.

Adım 5 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
▶          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

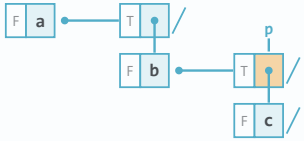
DEĞİŞKENLER

`p = &öğe(b)` `m = 0`

ÇAĞRI YIĞINI

- **derinlik(b)**
- derinlik(a)
- main()

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



Alt liste bulundu: derinliği özyinelemeli olarak hesaplanır.


```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
▶              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

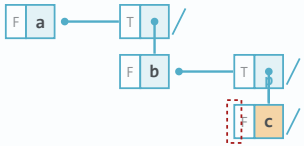
DEĞİŞKENLER

`p = &öğe((c))` `m = 0`

ÇAĞRI YIĞINI

- `derinlik(b)`
- `derinlik(a)`
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



3. düzey çağrı: `p = L` , `m = 0` .

Adım 7 / 16

```
1  int derinlik(oge_gost L) {
▶      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

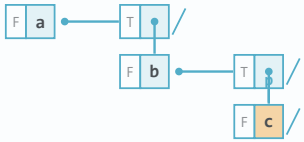
DEĞİŞKENLER

`p = &öğe(c)` `m = 0`

ÇAĞRI YIĞINI

- **derinlik(c)**
- derinlik(b)
- derinlik(a)
- main()

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



'c' atomik: derinliği artırmaz.

Adım 8 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
▶          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

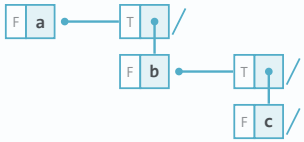
DEĞİŞKENLER

`p = &öğe(c)` `m = 0`

ÇAĞRI YIĞINI

- **derinlik(c)**
- derinlik(b)
- derinlik(a)
- main()

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



`p = p→bag` → NULL: bu düzeyin öğeleri bitti.

Adım 9 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
▶          p = p->bag;
10     }
11     return m + 1;
12 }
```

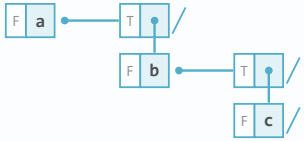
DEĞİŞKENLER

`p = NULL` `m = 0`

ÇAĞRI YIĞINI

- **derinlik(c)**
- derinlik(b)
- derinlik(a)
- main()

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



`return m + 1 = 1` : en derin alt liste + bu düzey.

Adım 10 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
▶      return m + 1;
12 }
```

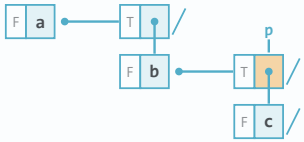
DEĞİŞKENLER

`p = NULL` `m = 0`

ÇAĞRI YIĞINI

- **derinlik(c)**
- derinlik(b)
- derinlik(a)
- main()

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



Alt listeden `n = 1` döndü → `m = 1`.

Adım 11 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
▶              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

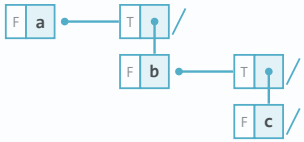
DEĞİŞKENLER

`p = &öğe((c))` `m = 1` `n = 1`

ÇAĞRI YIĞINI

- `derinlik(b)`
- `derinlik(a)`
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



`p = p→bag` → NULL: bu düzeyin öğeleri bitti.

Adım 12 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
▶          p = p->bag;
10     }
11     return m + 1;
12 }
```

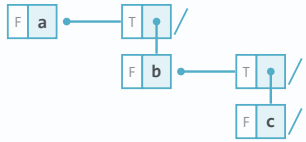
DEĞİŞKENLER

`p = NULL` `m = 1`

ÇAĞRI YIĞINI

- **derinlik(b)**
- derinlik(a)
- main()

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



`return m + 1 = 2` : en derin alt liste + bu düzey.

Adım 13 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
▶      return m + 1;
12 }
```

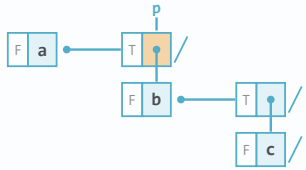
DEĞİŞKENLER

`p = NULL` `m = 1`

ÇAĞRI YIĞINI

- `derinlik(b)`
- `derinlik(a)`
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



Alt listeden `n = 2` döndü → `m = 2`.

Adım 14 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
▶              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
11     return m + 1;
12 }
```

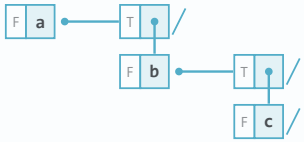
DEĞİŞKENLER

`p = &öğe((b,(c)))` `m = 2` `n = 2`

ÇAĞRI YIĞINI

- `derinlik(a)`
- `main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



`p = p→bag` → NULL: bu düzeyin öğeleri bitti.

Adım 15 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
▶          p = p->bag;
10     }
11     return m + 1;
12 }
```

DEĞİŞKENLER

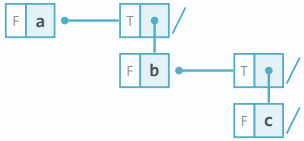
`p = NULL` `m = 2`

ÇAĞRI YIĞINI

`derinlik(a)`

`main()`

Kod İzleme: `derinlik(L)`, L = (a,(b,(c)))



`return m + 1 = 3` : en derin alt liste + bu düzey.

Adım 16 / 16

```
1  int derinlik(oge_gost L) {
2      oge_gost p = L;  int m = 0;     // m: en büyük
3      if (!L) return 0;
4      while (p) {
5          if (p->isaret) {
6              int n = derinlik(p->L_bag);
7              if (m < n) m = n;
8          }
9          p = p->bag;
10     }
▶      return m + 1;
12 }
```

DEĞİŞKENLER

`p = NULL` `m = 2`

ÇAĞRI YIĞINI

- `derinlik(a)`
- `main()`

SORU: derinlik — yinelemeli (örnek çözüm)

```
int derinlik( agac_gost L ){
    agac_gost yigit_al(); void yigit_ekle(agac_gost);
    int max = 0;
    for( int current = 1, p = L ; p ; p->bag ){
        if( p->isaret ){ // alt listeye gir
            yigit_ekle(p);
            p = p -> L_bag;
            current++;
        }
        if( !p->bag ){   // liste sonu, üste çık
            p = yigit_al();
            if( max < current ) max = current;
            current--;
        }
    }
    return max;
}
```

Derleme / döngü

`int current = 1, p = L` : p `int` olur; güncelleme `p→bag` atama değil → sonsuz döngü.

Uç durumlar

Boş alt listede NULL'a erişir; boş yığından alır; T'nin kardeşine geçmez.

Hocadan "Bu kod current, max değişkenleri tutularak döngüsel olarak yazılmayacak. Onu 1. sınıf öğrencisi yazar. … Ama global değişken kullanmak yasak!"

Yinelemeli derinlik: yerel yığınla

```
int derinlik_d(oge_gost L) {
    oge_gost yigit[100];       // yerel yığın (derinlik<100)
    int ust = 0, d = 1, m = 0;
    oge_gost p = L;
    if (!L) return 0;
    for (;;) {
        if (p == NULL) {             // düzey bitti
            if (ust == 0) return m;
            p = yigit[--ust]->bag;   // T'den sonraki
            d--;
        } else {
            if (d > m) m = d;
            if (p->isaret && p->L_bag) {
                yigit[ust++] = p;    // alt listeye in
                p = p->L_bag;
                d++;
            } else p = p->bag;
        }
    }
}
```

Yöntem

Alt listeye inerken T öğesi yığına konur; düzey bitince yığından alınır ve onun **kardeşine** geçilir.

Kurallar

Global değişken yok: yığın ve sayaçlar yerel. Boş alt liste ve boş liste ayrıca ele alındı.

Doğrulama

Altı örnek listede özyinelemeli sürümle aynı sonucu verdiği simülasyonla denetlendi.

DERİNLİK VE TERS ÇEVİRME

Özyinelemeli düşünme üzerine

Hocadan “Bana kalsaydı özyinelemeli düşünme altyapısını vermek için bu derse 3 değil 15 kredi önerirdim.”

Hocadan “Yemek yerken, yürürken, televizyon izlerken sürekli düşünmek gerekir: “Onu ne yaparsam ne döndürürüm?” … Debug yapılarak da nereye ne döndüğü görülmeli.”

Doğru çalışan kod “Verilen örnek üzerinde doğru çalışan, başka örneklerde çakılan kod yazma becerisi! Vallahi bravo! Hâlbuki kod farklı örnekler üzerinde, uç örnekler üzerinde de doğru çalışmalıdır.”

Orijinal kitap “Her dersten 2–3 kitap edinilmeli. Para verilip alınan kitap okunur; fotokopisi çektirilen okunmaz.”

Uç örnek listesi: (), (a), (()), ((())), tek düzeyli uzun liste, derin tek zincir.

SORU: paylaşımsız listeyi ters çevirme — L = (a,(b,c)) → ((c,b),a)

```
void devrik_al(oge_gost *L) {
    oge_gost r, p = *L, q = NULL;
    while (p) {
        if (p->isaret)              // atomik değilse
            devrik_al(&p->L_bag);
        r = q;
        q = p;
        p = p->bag;
        q->bag = r;
    }
    *L = q;
}
```

Ters çevirme yordamı + 2 satır

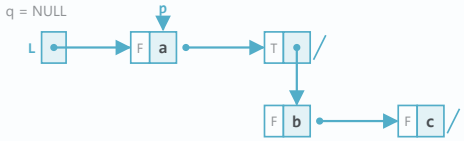
Tek yönlü bağlı listenin `liste_devrik_al` yordamındaki üç işaretçi aynen; ek: alt listeyse önce o ters çevrilir. Kullanım: görüntüyü yansıtmak.

Neden `&p→L_bag`?

Alt listenin başı değişir; T öğesinin `L_bag` alanı çağrıdan dönmeden, iç çağrının `*L = q` satırıyla güncellenir.

Hocadan "Böyle çözmezsek kaba kuvvet olurdu. Sonuçta o da devrik alıyor, ama çamları devirip devirip gidiyor."

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`p = *L` → &öğe(a), `q = NULL` .

Adım 1 / 12

```
1  void devrik_al(oge_gost *L) {
▶      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
11     *L = q;
12 }
```

DEĞİŞKENLER

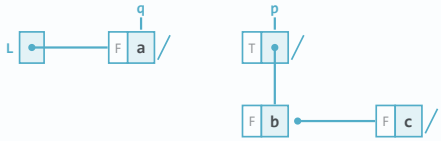
`r = ?` `q = NULL` `p = &öğe(a)` `*L = &öğe(a)`

ÇAĞRI YIĞINI

`devrik_al(&liste)`

`main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`r = q; q = p; p = p→bag; q→bag = r;` → a öğesinin bağı tersine çevrildi.

Adım 2 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
▶          r = q;
▶          q = p;
▶          p = p->bag;
▶          q->bag = r;
10     }
11     *L = q;
12 }
```

DEĞİŞKENLER

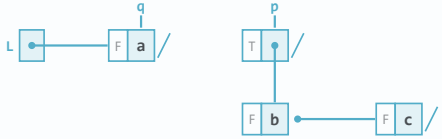
`r = NULL` `q = &öğe(a)` `p = &öğe((b,c))` `*L = &öğe(a)`

ÇAĞRI YIĞINI

`devrik_al(&liste)`

`main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



Öğe bir alt liste: önce o ters çevrilir (`&p→L_bag` adresiyle).

Adım 3 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
▶              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
11     *L = q;
12 }
```

DEĞİŞKENLER

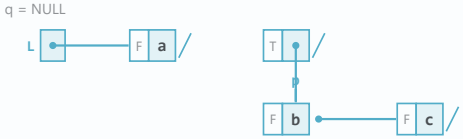
`r = NULL` `q = &öğe(a)` `p = &öğe((b,c))` `*L = &öğe(a)`

ÇAĞRI YIĞINI

`devrik_al(&liste)`

`main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`p = *L` → &öğe(b), `q = NULL`.

Adım 4 / 12

```
1  void devrik_al(oge_gost *L) {
▶      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
11     *L = q;
12 }
```

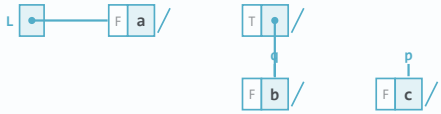
DEĞİŞKENLER

`r = ?` `q = NULL` `p = &öğe(b)` `*L = &öğe(b)`

ÇAĞRI YIĞINI

- `devrik_al(&p→L_bag)`
- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`r = q; q = p; p = p→bag; q→bag = r;` → b öğesinin bağı tersine çevrildi.

Adım 5 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
▶          r = q;
▶          q = p;
▶          p = p->bag;
▶          q->bag = r;
10     }
11     *L = q;
12 }
```

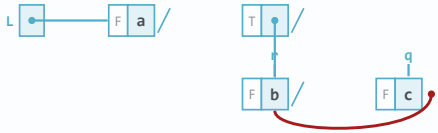
DEĞİŞKENLER

`r = NULL` `q = &öğe(b)` `p = &öğe(c)` `*L = &öğe(b)`

ÇAĞRI YIĞINI

- `devrik_al(&p→L_bag)`
- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`r = q; q = p; p = p→bag; q→bag = r;` → c öğesinin bağı tersine çevrildi.

Adım 6 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
▶          r = q;
▶          q = p;
▶          p = p->bag;
▶          q->bag = r;
10     }
11     *L = q;
12 }
```

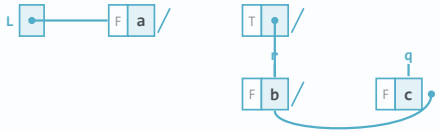
DEĞİŞKENLER

`r = &öğe(b)` `q = &öğe(c)` `p = NULL` `*L = &öğe(b)`

ÇAĞRI YIĞINI

- `devrik_al(&p→L_bag)`
- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`p == NULL` : bu düzey bitti.

Adım 7 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
▶      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
11     *L = q;
12 }
```

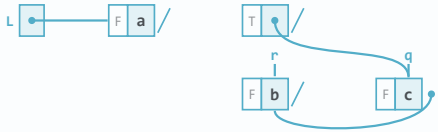
DEĞİŞKENLER

`r = &öğe(b)` `q = &öğe(c)` `p = NULL` `*L = &öğe(b)`

ÇAĞRI YIĞINI

- `devrik_al(&p→L_bag)`
- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`*L = q` : bu düzeyin yeni başı c.

Adım 8 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
▶      *L = q;
12 }
```

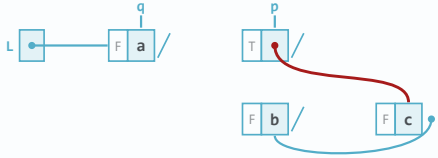
DEĞİŞKENLER

`r = &öğe(b)` `q = &öğe(c)` `p = NULL` `*L = &öğe(c)`

ÇAĞRI YIĞINI

- `devrik_al(&p→L_bag)`
- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



Alt listeden dönüldü: T öğesinin `L_bag` alanı zaten yeni başı gösteriyor.

Adım 9 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
▶              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
11     *L = q;
12 }
```

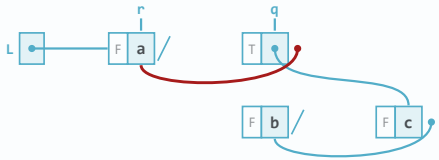
DEĞİŞKENLER

`r = NULL` `q = &öğe(a)` `p = &öğe((c,b))` `*L = &öğe(a)`

ÇAĞRI YIĞINI

- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`r = q; q = p; p = p→bag; q→bag = r;` → (c,b) öğesinin bağı tersine çevrildi.

Adım 10 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
▶          r = q;
▶          q = p;
▶          p = p->bag;
▶          q->bag = r;
10     }
11     *L = q;
12 }
```

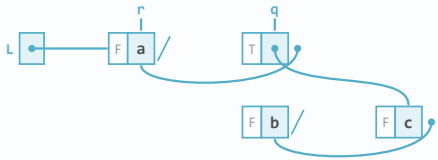
DEĞİŞKENLER

`r = &öğe(a)` `q = &öğe((c,b))` `p = NULL` `*L = &öğe(a)`

ÇAĞRI YIĞINI

- `devrik_al(&liste)`
- `main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`p == NULL` : bu düzey bitti.

Adım 11 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
▶      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
11     *L = q;
12 }
```

DEĞİŞKENLER

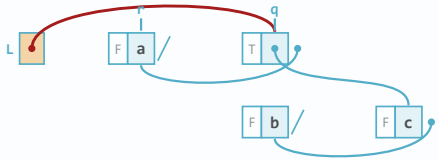
`r = &öğe(a)` `q = &öğe((c,b))` `p = NULL` `*L = &öğe(a)`

ÇAĞRI YIĞINI

`devrik_al(&liste)`

`main()`

Kod İzleme: `devrik_al(&L)`, (a,(b,c)) → ((c,b),a)



`*L = q` : bu düzeyin yeni başı (c,b).

Adım 12 / 12

```
1  void devrik_al(oge_gost *L) {
2      oge_gost r, p = *L, q = NULL;
3      while (p) {
4          if (p->isaret)              // atomik değilse
5              devrik_al(&p->L_bag);
6          r = q;
7          q = p;
8          p = p->bag;
9          q->bag = r;
10     }
▶      *L = q;
12 }
```

DEĞİŞKENLER

`r = &öğe(a)` `q = &öğe((c,b))` `p = NULL` `*L = &öğe((c,b))`

ÇAĞRI YIĞINI

`devrik_al(&liste)`

`main()`

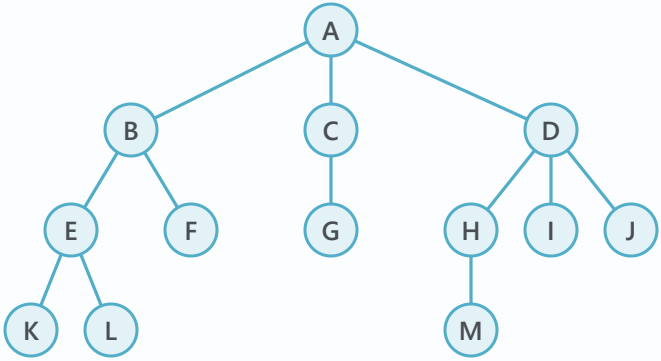
Veri yapıları: bel kemiği

İş görüşmesi “Veri yapıları dersini aldın mı?” “Evet, aldım.” “Ben de aldım. Şu list, tree falan değil mi?” … Aynı şunun gibi: “Beyin cerrahisi? Ha, şu damar değil mi?” Sanki o kadar basitti. Veri yapıları, bilgisayar bilimlerinin bel kemiğidir. List'i tree'yi bilmek, veri yapısını öğrenmiş olmak değil. Bunu yapan yordamın yazılabilmesi gerekir.

Okuma Parçası: Teorik Arka Plan ve Matematik “Bizde öğrenci 1. sınıfa başlar başlamaz programlama ödevleri veriliyor… avantajı hızlı kod yazabilme ve hızlı iş bulabilme. Ama dezavantajı da var; teoriklere önem verilmiyor… Oysa çok iyi matematiksel arka plan gerekiyor: Discrete, istatistik, stokastik süreçler… Richard Johnsonbaugh – Discrete Mathematics: mutlaka alınıp okunmalı. Okunmazsa derslerden geçilir, ama bu iş öğrenilemez.”

Okuma Parçası “Bölüm kısıtlıdır, her şeyi veremez! Öğrenci daha 2. sınıftayken bunu kendisi fark edip matematik bölümünden, istatistik bölümünden… ders almalı.”

Ağaç yapısının genelleştirilmiş listeyle kurulması




```
A(B(E(K,L),F),C(G),D(H(M),I,J))
```

Notasyon

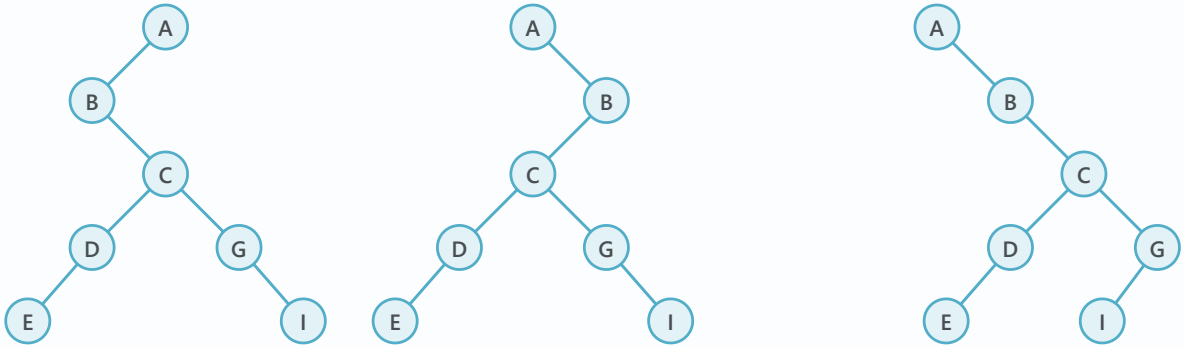
`C(G)` : C, G'nin ebeveynidir.
`A(B…,C…,D…)` : A; B, C, D'nin ebeveynidir.

Esneklik

Bu kadar veri için tek satırlık metin yeter. Virgül yerine nokta, süslü parantez: notasyon seçime kalmış.

Hocadan "Bundan ona, ondan buna geçilebiliyorsa tamamdır; içerideki yapıyı kimse bilemez."

Dezavantaj: sağ–sol ayrımı yok



Üç ağacın da doğrusal formu aynı: A(B(C(D(E),G(I)))) — B'nin sol mu sağ çocuk olduğu yazılamıyor.

Sorun

Genel ağaçta çocuk sırası yeterlidir; ama ikili ağaçta **sol** ve **sağ** çocuk ayrımı kaybolur.

Çözüm

Her düğümün yanında L/R bilgisi tutulur (ya da boş çocuk yerine özel simge yazılır): A(B(,C(…))).

Ödevler

1. **Ödev 1:** `A(B(E(H,I(J,K)),F),C(G),D)` listesinin bellek görünümünü ve ağaç yapısını çiziniz.
2. **Ödev 2:** Ağaç yapısı verildiğinde liste yazımını çıktı veren yordamı yazınız.

▶ **Çözüm önerileri**

Paylaşımlı listeler ve referans sayma

```
// başlıkta sayac: bu listeyi gösteren sayısı
void liste_birak(oge_gost *L) {
    oge_gost p, q;
    if (*L == NULL) return;
    if (--(*L)->sayac > 0) {    // başkası kullanıyor
        *L = NULL; return;
    }
    p = (*L)->bag;              // ilk gerçek öğe
    free(*L);
    while (p) {
        if (p->isaret) liste_birak(&p->L_bag);
        q = p; p = p->bag; free(q);
    }
    *L = NULL;
}
```

Başlık düğümü

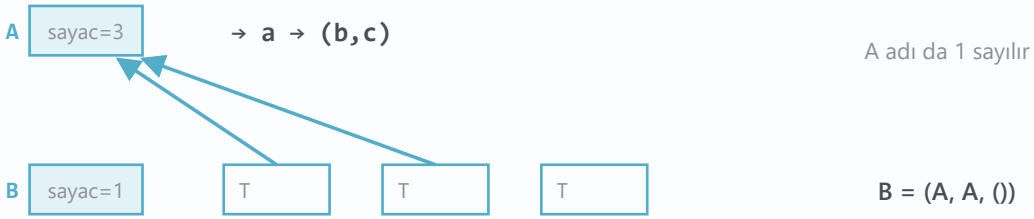
Her liste bir başlık düğümüyle başlar; `sayac` onu gösteren işaretçi sayısıdır.

Bırakma kuralı

Sayaç 0'a inmedikçe liste bırakılmaz; inerse öğeler ve alt listeler de aynı kuralla bırakılır.

Horowitz–Sahni, Bölüm 4.9 (referans sayaçlı başlık düğümü).

Referans sayma adım adım: B = (A, A, ())



Her listenin başında bir **başlık düğümü**; `sayac` onu gösteren işaretçi sayısı. A'yı A adı ve B'nin iki öğesi gösterir: 3.

Adım 1 / 4

Kopyalamada da aynı fikir: paylaşımlı alt liste yeniden kopyalanmaz, sayacı artırılır.

Referans sayma adım adım: B = (A, A, ())



`liste_birak(&B)` : B'nin sayacı 1 → 0, B'nin öğeleri tek tek bırakılır.

Adım 2 / 4

Kopyalamada da aynı fikir: paylaşımlı alt liste yeniden kopyalanmaz, sayacı artırılır.

Referans sayma adım adım: B = (A, A, ())

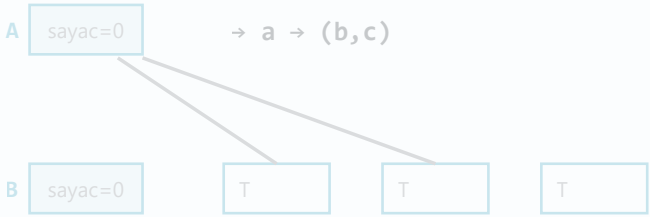


B'nin iki T öğesi A için `liste_birak` çağırır: 3 → 2 → 1. A hâlâ kullanımda, bırakılmaz.

Adım 3 / 4

Kopyalamada da aynı fikir: paylaşımlı alt liste yeniden kopyalanmaz, sayacı artırılır.

Referans sayma adım adım: B = (A, A, ())



Sonra `liste_birak(&A)`: 1 → 0, A bırakılır. Paylaşım güvenle yönetildi.

Adım 4 / 4

Kopyalamada da aynı fikir: paylaşımlı alt liste yeniden kopyalanmaz, sayacı artırılır.

Referans saymanın sınırı ve çöp toplama

Döngü sorunu

C = (a, C): C kendini gösterdiği için sayacı hiç 0 olmaz; C adı silinse bile liste bellekte kalır (sızıntı).

İşaretle–süpür

(1) Kök işaretçilerden (değişkenler) erişilebilen tüm öğeler **işaretlenir**; (2) bellek taranır, işaretsizler **süpürülür** (avail'e eklenir).

Bedel

Çöp toplayıcı çalışırken program durur; işaret biti için yer gerekir. LISP, Java, Python bu fikrin türevlerini kullanır.

C'de

Otomatik çöp toplama yok: kopyalama, paylaşım ve bırakma sorumluluğu programcıda.

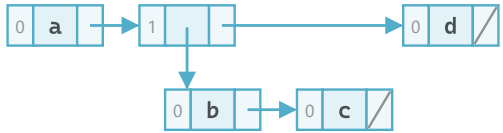
Laboratuvar: Liste Derinliği

(a, (b, c), d)

Girdi (liste, ör. (a, (b, c), d)) `(a, (b, c), d)` Uygula

KOD

```
 1 int derinlik(oge_gost L) {
 2     oge_gost p = L;  int m = 0;  // en büyük
 3     if (!L) return 0;
 4     while (p) {
 5         if (p->isaret) {
 6             int n = derinlik(p->L_bag);
 7             if (m < n) m = n;
 8         }
 9         p = p->bag;
10     }
11     return m + 1;
12 }
```



çağrı derinliği 0

ÇIKTI (henüz yok)

`derinlik(L)` : her düzey için özyinelemeli bir çağrı açılır. Etiketi 1 olan öğe bir alt liste gösterir; o alt listenin derinliği ayrıca hesaplanır.

KAPANIŞ

Sınav sorusu

Soru. Öğeye `int sayac` alanı eklendi. İşaret alanı TRUE olan her öğenin sayacına altındaki öğe sayısını yazan ve toplam öğe sayısını döndüren yordamı **özyinelemeli** yazınız.

▶ **Örnek çözüm**

KAPANIŞ

Sınav sorusu

Soru. Bellekte bulunan genelleştirilmiş listeyi **doğrusal formda** yazdıran özyinelemeli yordamı yazınız.

▶ **Çözüm**

KAPANIŞ

Sınav sorusu

Soru. `L=(d,(c,(e,(m,k),(()),z,y,(x,(r),b,a,()))))` doğrusal gösterimine karşılık gelen genelleştirilmiş liste görünümünü çiziniz. Aynı düzeyde peşpeşe gelen öğeler bir satır boyunca izlenebilmelidir.

▶ **Çözüm**

KAPANIŞ

Karmaşıklık özeti

Yordam	Zaman	Ek bellek	Not
kopyala	O(n)	O(derinlik + uzunluk)*	paylaşımsız, özyinelemesiz
esitmi / esit	O(n)	O(derinlik + uzunluk)*	ilk farkta durur
dolas_yaz	O(n)	O(derinlik + uzunluk)*	preorder sırası
derinlik (özyinelemeli)	O(n)	O(derinlik)	döngü düzeyi, özyineleme alt liste
derinlik_d (yinelemeli)	O(n)	O(derinlik)	yerel yığın
devrik_al	O(n)	O(derinlik)	yerinde, malloc yok
dizi_ekle / dizi_sil	O(n)	O(1)	kaydırma
bl_ekle (konum i)	O(i)	O(1)	kaydırma yok

n: toplam öğe sayısı. * `bag` üzerinden de özyinelemeli çağrı yapan yordamlarda yığın derinliği bir düzeydeki öğe sayısına da bağlıdır; `bag` için döngü kullanmak bunu O(derinlik)'e indirir.

KAPANIŞ

Sık yapılan hatalar

1. Paylaşımlı listeyi paylaşımsız varsayıp kopyalamak ya da iki kez bırakmak.
2. Özyinelemeli listede (C = (a,C)) kopyala/eşitlik çağırmak: sonsuz özyineleme.
3. `if`'lerde süslü parantezi unutup NULL için de çağrı yapmak (`dolas_yaz`).
4. Baş değişen yordama listeyi değerle vermek (ters çevirme `oge_gost *L` ister).
5. Boş alt liste `()` ile boş liste NULL'ı karıştırmak; T öğesi var, L_bag NULL.
6. Global değişkenle derinlik saymak; yordam başka bir bileşenin parçası olunca bozulur.

KAPANIŞ

Alıştırmalar

1. Genelleştirilmiş listedeki atom sayısını bulan yordamı yazın.
2. Doğrusal form metnini (`"(a,(b,c))"`) okuyup listeyi kuran yordamı yazın.
3. `kopyala` paylaşımı koruyacak biçimde nasıl değiştirilir?
4. Bir x atomunun listede geçip geçmediğini bulan `icerir(L, x)` yazın.
5. Dizi tabanlı listeyi dolunca iki katına büyüyen biçimde yeniden yazın.

▶ **Cevaplar / ipuçları**

Hocadan "Listeyi meydana getiren yordam, quizlik soru! Listeyi kuran yordamı nasıl yazarım, kopyalayanı nasıl yazarım diye sürekli düşünmek gerekir!"

KAPANIŞ

Bölüm özeti

Liste ADT

Sözleşme sabit, gerçekleme değişebilir: dizi hızlı erişim, bağ hızlı ekleme-silme.

Genelleştirilmiş liste

Öğe atom ya da alt liste: [isaret | veri/L_bag | bag].

Özyinelemeli kalıp

Atomsa işlenir, alt listeyse özyinelemeyle inilir, sonra `bag` ile devam edilir.

Derinlik ve ters çevirme

Özyinelemeli yazım kısa ve doğru; yinelemeli yazım yerel yığın ister.

Ağaç

Tek satırlık doğrusal form; ikili ağaçta sol/sağ bilgisi eklenmeli.

Paylaşım

Kopya ve bırakmada tuzak; referans sayma döngüde yetmez, çöp toplama gerekir.

KAPANIŞ

Kaynaklar ve devam

- E. Horowitz, S. Sahni, S. Anderson-Freed, *Fundamentals of Data Structures in C*, 2. bs., Bölüm 4.9 (genelleştirilmiş listeler, referans sayma).
- T. H. Cormen vd., *Introduction to Algorithms*, Bölüm 10 (liste gerçeklemeleri).
- R. Johnsonbaugh, *Discrete Mathematics* (Okuma Parçası önerisi).

← Bölüm 8: Yığın ve Kuyruk Uygulamaları	Ders sayfası	Bölüm 10: Ağaçlar

Tek yönlü bağlı liste yordamları ve `serbest_birak` sınav sorusu: Bağlı Listeler.

Son

Bölüm 9: Liste ADT ve Genelleştirilmiş Listeler

Sonraki bölüm: Bölüm 10: Ağaçlar · Ders sayfası